You are assigned to develop a **Hotel Management System** for a hotel to manage rooms, customers, and billing. The system should calculate the **final bill amount** for a customer by considering:

- Models hotel rooms and customers
- Calculates base room price based on stay duration
- Applies **seasonal** and **loyalty** discounts
- Calculates and displays the **final bill amount**

Files to Complete

You must complete the following files:

1. src/main/java/com/edutech/Room.java
2. src/main/java/com/edutech/Customer.java
3. src/main/java/com/edutech/HotelManagementSystem.java
4. src/main/java/com/edutech/Main.java

Class Requirements

1. Room Class

Purpose:

Represents a room available in the hotel.

Fields (Private):

- int roomNumber
- String roomType
- double pricePerNight

Constructor:

Room(int roomNumber, String roomType, double pricePerNight)

Methods:

- Getter and setter methods for all fields

2. Customer Class

Purpose:

Represents a customer staying in the hotel.

Fields (Private):

- String customerName
- boolean isLoyalCustomer

Constructor:

Customer(String customerName, boolean isLoyalCustomer)

Methods:

- Getter and setter methods for all fields

3. HotelManagementSystem Class

Purpose:

Handles room management, customer management, and billing logic.

Fields:

- List<Room> rooms
- List<Customer> customers

Constructor:

- Initializes both lists

Methods to Implement:

- void addRoom(Room room)
- void removeRoom(Room room)
- void addCustomer(Customer customer)
- double calculatePrice(Room room, int numberOfNights)
    - Returns base price using:
    - pricePerNight × numberOfNights
- double applyDiscount(Customer customer, Month currentMonth)
    - Returns total discount percentage
- double calculateFinalBill(Room room, int numberOfNights, Customer customer, Month currentMonth)

- Applies discount and returns final bill amount

Discount Rules

Seasonal Discount

- **Off-Season (December – February):** 15%
- **Shoulder Season (March – May, September – November):** 10%
- **Peak Season (June – August):** No discount

Loyalty Discount

- Loyal customers receive an **additional 5% discount**
- Non-loyal customers receive **no loyalty discount**

Total Discount Formula

Total Discount = Seasonal Discount + Loyalty Discount

Final Bill Formula

Base Price = pricePerNight × numberOfNights

Final Bill = Base Price × (1 − Total Discount)

4. Main Class

Purpose:

Acts as the entry point of the application.

Main Method Should:

- Create room objects
- Create customer objects
- Calculate and display final bills for different scenarios

Constraints

- Use **only in-memory storage** (no database)
- Follow proper **encapsulation**

- Discount logic must match the given rules
- Method signatures must not be modified

Notes for Learners

- Use java.time.Month for seasonal calculations
- Ensure discount calculation is done **before final bill computation**
- Keep the code readable and modular
- Avoid hardcoding values outside the specified rules

****

Sample input & output:

```
(335 kB at 658 kB/s)
Calculating bill for John Doe staying in a Single room for 3 nights (Winter/Off-season):
Final Bill: $240.00

Calculating bill for Jane Smith staying in a Suite for 3 nights (Peak Season):
Final Bill: $900.00

Calculating bill for John Doe staying in a Double room for 3 nights (Shoulder Season):
Final Bill: $382.50
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  6.551 s
[INFO] Finished at: 2025-11-21T04:33:00Z
[INFO] ------------------------------------------------------------------------
* Terminal will be reused by tasks, press any key to close it.
```

PROBLEM 2

You are working on a Human Resource Management System (HRMS) where you need to filter out employees based on their performance scores and years of experience.

The goal is to filter employees who have a performance score greater than or equal to a certain threshold and years of experience above a certain value.

Implement using Simple Linked List

Your Task:

1. **Employee class**:
Fields:
int employeeID, String name, double performanceScore, int yearsOfExperience

Create Constructor and Getters, Setter and toString()

2. **EmployeePerformanceApp.java**:

a. filterEmployeesByPerformanceAndExperience(List<Employee> employees, double minPerformanceScore, int minExperience);
Parameters:
- employees: A list of Employee objects, each containing fields like employeeID, name, performanceScore, and yearsOfExperience.
- minPerformanceScore: A double representing the minimum performance score an employee must have to be included in the list.
- minExperience: An integer representing the minimum years of experience an employee must have to be included in the list.

Return Value:
- A list of Employee objects who have a performance score greater than or equal to minPerformanceScore and years of experience greater than or equal to minExperience.

b. main(String[] args);
- This method should create a list of Employee objects.
- It should call filterEmployeesByPerformanceAndExperience() method to filter the employees based on the performance score 80.0 and experience 5 Years.
- Pass default values to the Employee Objects

- Filtering employees based on performance and experience
- Finally, it should print the filtered list of employees.

Application Requirements / Notes:

- An Employee class is assumed to have the following fields: employeeID, name, performanceScore, and yearsOfExperience. (Employee class code is already provided).
- The program should correctly handle edge cases, such as no employees meeting the criteria or all employees passing the filter.
- Implement using Simple Linked List

Sample Input and output

```
Downloaded from central: https://repo.maven.apache.org/maven2/commons-codec/commons-codec/1.11/common
Employee{employeeID=1, name='John Doe', performanceScore=85.5, yearsOfExperience=5}
Employee{employeeID=2, name='Jane Smith', performanceScore=92.3, yearsOfExperience=8}
Employee{employeeID=4, name='Bob Johnson', performanceScore=88.9, yearsOfExperience=6}
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  11.050 s
[INFO] Finished at: 2025-08-08T09:01:55Z
```

PROBLEM 3

You are working on a Human Resource Management System (HRMS) where you need to filter out employees based on their performance scores and years of experience.

The goal is to filter employees who have a performance score greater than or equal to a certain threshold and years of experience above a certain value.

Implement using Simple Linked List

Your Task:

1. **Employee class**:
Fields:
int employeeID, String name, double performanceScore, int yearsOfExperience

Create Constructor and Getters, Setter and toString()

2. **EmployeePerformanceApp.java**:

a. filterEmployeesByPerformanceAndExperience(List<Employee> employees, double minPerformanceScore, int minExperience);
Parameters:
- employees: A list of Employee objects, each containing fields like employeeID, name, performanceScore, and yearsOfExperience.
- minPerformanceScore: A double representing the minimum performance score an employee must have to be included in the list.
- minExperience: An integer representing the minimum years of experience an employee must have to be included in the list.

Return Value:
- A list of Employee objects who have a performance score greater than or equal to minPerformanceScore and years of experience greater than or equal to minExperience.

b. main(String[] args);
- This method should create a list of Employee objects.
- It should call filterEmployeesByPerformanceAndExperience() method to filter the employees based on the performance score 80.0 and experience 5 Years.
- Pass default values to the Employee Objects

- Filtering employees based on performance and experience
- Finally, it should print the filtered list of employees.

Application Requirements / Notes:

- An Employee class is assumed to have the following fields: employeeID, name, performanceScore, and yearsOfExperience. (Employee class code is already provided).
- The program should correctly handle edge cases, such as no employees meeting the criteria or all employees passing the filter.
- Implement using Simple Linked List

Sample Input and output

```
Downloaded from central: https://repo.maven.apache.org/maven2/commons-codec/commons-codec/1.11/common
Employee{employeeID=1, name='John Doe', performanceScore=85.5, yearsOfExperience=5}
Employee{employeeID=2, name='Jane Smith', performanceScore=92.3, yearsOfExperience=8}
Employee{employeeID=4, name='Bob Johnson', performanceScore=88.9, yearsOfExperience=6}
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  11.050 s
[INFO] Finished at: 2025-08-08T09:01:55Z
```

PROBLEM 4

You are building an Event management application. The application must contain the classes Event and Attendee with fields like ID and Name. Store all attendees in a Set to ensure that an attendee with the same ID doesn't get registered twice.

Requirements:

Attendee.java:

Define the following fields and methods in the Attendee class:

Variables:

1. int id
2. String name

Methods:

- Define constructor and Getter and Setter methods.
- Override equals() and hashCode() to prevent duplicate IDs
- Override toString() to print the Attendee details

Event.java:

Define the following fields and methods in the Event class:

1. **Set<Attendee> attendees** - private field to store the attendees.
2. **Event()** : A default empty constructor that initializes the attendees set as an empty HashSet.
3. **public boolean addAttendee(Attendee attendee):** a method that adds the attendee to the attendees set and returns true if the attendee is successfully added. If the attendee is already present in the set, return false.
4. **public boolean removeAttendee(Attendee attendee)** : a method that removes the attendee from the attendees set and returns true if the attendee is successfully removed. If the attendee is not present in the set, return false.
5. **public int getTotalAttendees():** a method that returns the total number of attendees in the set.

EventManagementApp.java

1. create an Event object. create new Attendees. Pass the Attendee details like id and name through constructor and add the Attendee object to the Event entity.
2. Print all the attendees to the event in the following message format:"**Total attendees: "**
3. Remove one attendee from the event
4. Print the total attendees in the following message format:" **Total attendees after removal:"**

Note: Users need to pass the default values to "**Attendee**" Entity.

Sample Input/Output Screenshots:

```
Total attendees: 3
Total attendees after removal: 2
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  11.869 s
[INFO] Finished at: 2025-08-10T15:52:33Z
[INFO] ------------------------------------------------------------------------
```

PROBLEM 5

You are tasked with developing an inventory management system for a fruit store. The system should allow the store to add different types of fruits along with their quantities to the inventory and retrieve the quantity of a specific fruit when requested.

Functionality Requirements:

Initialization:

- The system should initialize an empty inventory using a HashMap where the key is the fruit name (a string) and the value is the quantity (an integer).

Add Fruit:

- The system should allow adding a specified quantity of a fruit to the inventory.
- **Method:** static void addFruit(String fruitName, int quantity)
- **Description:** Adds the specified quantity of the given fruit to the inventory. If the fruit already exists in the inventory, its quantity is updated.

Read Fruit:

- The system should allow retrieving the quantity of a specified fruit from the inventory.
- **Method:** static Integer readFruit(String fruitName)
- **Description:** Retrieves the quantity of the specified fruit from the inventory. If the fruit does not exist in the inventory, it returns null.

Main Method:

- The main method should handle user input to add fruits to the inventory and retrieve the quantity of a specific fruit.
- It then prints the current inventory and the quantity of the requested fruit.

Input Format:

- The first line contains an integer, n, representing the number of fruits to be added to the inventory.
- Read a string (fruit name) and an integer (quantity) n times
- The next line contains a string representing the name of the fruit whose quantity is to be retrieved (**refer screenshot**).

Output Format:

- The system should print the current inventory in the format {fruitName=quantity, ...}.
- The system should print the quantity of the requested fruit in the format fruitName: quantity. If the fruit does not exist in the inventory, it should print fruitName: null.

Constraints:

- 1 <= n <= 100
- 1 <= quantity <= 1000
- fruitName is a string with a maximum length of 50 characters.

Example:

Input:

3
Apple
50
Banana
30
Orange
20
Banana

Output:

{Apple=50, Banana=30, Orange=20}
Banana: 30

Sample Input/Output Screenshot:

```
11/commons-codec-1.11.jar (335 kB at 338 kB/s)
3
Apple
20
mango
40
Banana
25
mango
{Apple=20, mango=40, Banana=25}
mango: 40
[INFO] ----------------------------------------------------
```

PROBLEM 6

Develop a movie rating system using a LinkedHashMap to store movie names and their ratings. The system should allow users to add movies with their ratings, ensuring that the insertion order is maintained. Implement functionality to display movie names in ascending order and retrieve the rating of a specific movie. If a movie is not found, return an appropriate message.

MovieRatingSystem.java:

Create a LinkedHashMap named movies as given below:

static LinkedHashMap<String, Double> movies = new LinkedHashMap<>();

Include the below methods:

1. Add Movies: The system should allow adding movies with their ratings.
2. Display Movies in Ascending Order: The system should display movie names in ascending order.
3. Get Movie Rating: The system should retrieve the rating of a specific movie.
4. Get Movies: Listout all the movies

Method Signatures:

public static void **addMovie(String movieName, double rating)**;
public static String **getMovie(String movieName)**; - return the rating of the movie or "Movie not found" message
public static Map<String, Double> **getMovies()**; - Return movies list
public static void **displayMovieNameASCOrder();**

Main method:

1. The first line contains an integer, n, the number of movies to add.
2. The next n lines each contain the details of a movie in the following format and add to the movies:
   a. Movie Name (String)
   b. Rating (Double)
3. print the movie name and rating (**refer screenshot**).
4. Read the name of the movie to retrieve the rating and get the rating-Print the rating of the specified movie or "Movie not found" if the movie does not exist.
5. At last, print all movie names in ascending order

Sample Input

3
Inception
8.8
The Dark Knight
9.0
Interstellar
8.6
Inception

Sample Output

{Inception=8.8, The Dark Knight=9.0, Interstellar=8.6}
Movie rating: 8.8
[Inception, Interstellar, The Dark Knight]

Constraints

- The Movie Name is a non-empty string.
- The Rating is a double between 0.0 and 10.0.
- The number of movies n is between 1 and 100.

Sample Input/Output Screenshot:

```
[INFO]
[INFO] --- exec-maven-plugin:3.0.0:java (default-cli) @ solution ---
3
Enter Movie 1:
Movie Name: Shole
Rating: 8.8
Enter Movie 2:
```

```
Movie Name: Divana ho gaye
Rating: 8.5
Enter Movie 3:
Movie Name: Kish
Rating: 9.9
{Shole=8.8, Divana ho gaye=8.5, Kish=9.9}
Movie Name: Divana1
Movie not found
[Divana ho gaye, Kish, Shole]
[INFO] ------------------------------------------------------------------------
[INFO] mBUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:08 min
```

PROBLEM 7

You are tasked with creating a leaderboard system for a multiplayer game using a **TreeMap.** The leaderboard should maintain a ranking of players based on their scores.

Leaderboard.java:

Create a leaderboard instance:

static TreeMap<Integer, String> leaderboard= new TreeMap<>(Collections.reverseOrder());

Implement the following functionalities:

1. Add Player: Add a player with a given score to the leaderboard. Method: public void addPlayer(int score,String playerName)
2. Delete Player: Delete a player from the leaderboard. Method: public void deletePlayer(String playerName)
3. Display Top Players: Display the top n players from the leaderboard. Method: public TreeMap<Integer, String> displayTopPlayers(int n)

Main method:

create a new LeaderBoard object

Input:

· The first line contains an integer, n, representing the number of players to be added to the collection.
· The next two lines each contain the players scores and name and add to the leaderboard object
print the Current leaderboard
Next read the name of the player to delete and call the respective delete method and display the Updated leaderboard
Next read the number of top players to display, call the display method and print the details

Constraints

- Scores should be unique.
- Name should be unique.
- Scores are non-negative integers.
- 1 ≤ n ≤ number of players in the leaderboard

Sample Input/Output Screenshot:

```
Enter number of players
3
Enter score
200
Enter name
```

```
Enter score
200
Enter name
akal
Enter score
160
Enter name
sara
Enter score
500
Enter name
rakshi
Current leaderboard:
{500=rakshi, 200=akal, 160=sara}
Enter the name of the player to delete:
rakshi
Player rakshi removed successfully.
Updated leaderboard:
{200=akal, 160=sara}
Enter the number of top players to display:
2
{200=akal, 160=sara}
```

```
Enter number of players
-3
Current leaderboard:
{}
Enter the name of the player to delete:
2
Player 2 not found.
Updated leaderboard:
{}
Enter the number of top players to display:
```

PROBLEM 8

You are tasked with creating a system to analyze stock prices using a TreeMap. The system should be able to add stock prices and fetch the second minimum stock price. Implement the following functionalities:-

StockPriceAnalysis.java:

Declare and initialize through the constructor:

private Map<String, Double> symbolToPrice;

private TreeMap<Double, Integer> priceCount;

Add Stock:

method: public void addStock(String symbol, double price)

Add a stock with a given symbol and price to the system. The symbol is a unique identifier for the stock, and the price is a non-negative double representing the stock's current price. If symbol already exists, decrement previous price count

Get Second Minimum Price:

Method: public String getSecondMinimumPrice()

Retrieve the second lowest stock price from the system. If there are fewer than two stocks in the system, return a message "**Not enough elements to find the second minimum price.**"

Main method:

create an instance of StockPriceAnalysis

Input Format

- The first line contains an integer, n, representing the number of Stock to be added to the collection.
- The next lines read the stock name and price and add to StockPriceAnalysis object (**refer screenshot**).
- Call the method secondMinimumPrice()
- Add one more stock and call the method secondMinimumPrice()

Output Format

- print the second lowest stock price. If there are fewer than two stocks, print "Not enough elements to find the second minimum price."

Constraints

- Stock symbols are unique.
- 1 ≤ number of operations ≤ 1000

Sample Input/Output Screenshot:

```
3
AAPL
150.0
GOOGL
120.0
MSFT
180.0
150.0
AMZN
140.0
140.0
[INFO] ------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------
[INFO] Total time:  51.328 s
[INFO] Finished at: 2025-11-24T09:09:19Z
```

Sample Test Case Screenshot:

JDBC

PROBLEM 1

A company wants to send out a marketing campaign to all its customers in the United States. The company wants to sort the list of customers by name in descending order, so that it can start with its most important customers. The company also wants to include the customer's city and country in the list, so that it can personalize the marketing campaign.

Task:

- Write a SQL query to find all customers in the "USA" sorted by CustomerName in descending order, along with their City and Country.

Customers Table Column details:

- CustomerID, CustomerName, ContactName, Address, City, PostalCode, Country

Hints

Use SELECT Query

Constraints

None

Sample Input 1

```
CREATE TEMPORARY TABLE Customers
(
    CustomerID INTEGER PRIMARY KEY AUTO_INCREMENT,
    CustomerName VARCHAR(50),
    ContactName VARCHAR(50),
    Address VARCHAR(50),
    City VARCHAR(20),
    PostalCode VARCHAR(10),
    Country VARCHAR(15)
);

INSERT INTO Customers VALUES(94, 'Taj Mahal Textiles', 'Amit Kumar', '789 Fabric Road', 'Bangalore', '560001', 'India');
INSERT INTO Customers VALUES(95, 'Mumbai Electronics', 'Neha Verma', '101 Circuit Avenue', 'Mumbai', '400002', 'India');
INSERT INTO Customers VALUES(96, 'Chennai Furniture Mart', 'Arvind Singh', '234 Sofa Street', 'Chennai', '600001', 'India');
INSERT INTO Customers VALUES(1,'Alfreds Futterkiste','Maria Anders','Obere Str. 57','Berlin','12209','Germany');
INSERT INTO Customers VALUES(2,'Ana Trujillo Emparedados y helados','Ana Trujillo','Avda. de la Constitución 2222','Mexico D.F.','5021','Mexico');
INSERT INTO Customers VALUES(3,'Antonio Moreno Taqueria','Antonio Moreno','Mataderos 2312','Mexico D.F.','5023','Mexico');
```

```
INSERT INTO Customers VALUES(4,'Around the Horn','Thomas Hardy','120 Hanover Sq.','London','WA1 1DP','UK');
INSERT INTO Customers VALUES(5,'Berglunds snabbkp','Christina Berglund','Berguvsvägen 8','Luleå','S-958 22','Sweden');
INSERT INTO Customers VALUES(50,'Maison Dewey','Catherine Dewey','Rue Joseph-Bens 532','Bruxelles','B-1180','Belgium');
INSERT INTO Customers VALUES(51,'Mère Paillarde','Jean Fresnière','43 rue St. Laurent','Montréal','H1J 1C3','Canada');
INSERT INTO Customers VALUES(52,'Morgenstern Gesundkost','Alexander Feuer','Heerstr. 22','Leipzig','4179','Germany');
INSERT INTO Customers VALUES(45,'Let''s Stop N Shop','Jaime Yorres','87 Polk St. Suite 5','San Francisco','94117','USA');
INSERT INTO Customers VALUES(43,'Lazy K Kountry Store','John Steel','12 Orchestra Terrace','Walla Walla','99362','USA');
INSERT INTO Customers VALUES(36,'Hungry Coyote Import Store','Yoshi Latimer','City Center Plaza 516 Main St.','Elgin','97827','USA');
```

Sample Output 1

```
CustomerName,City,Country
Let's Stop N Shop,San Francisco,USA
Lazy K Kountry Store,Walla Walla,USA
Hungry Coyote Import Store,Elgin,USA
```

Sample Input 2

```
CREATE TEMPORARY TABLE Customers
(
    CustomerID INTEGER PRIMARY KEY AUTO_INCREMENT,
    CustomerName VARCHAR(50),
    ContactName VARCHAR(50),
    Address VARCHAR(50),
    City VARCHAR(20),
    PostalCode VARCHAR(10),
    Country VARCHAR(15)
);


INSERT INTO Customers VALUES(24,'Folk och fa HB','Maria Larsson','Åkergatan 24','Bräcke','S-844 67','Sweden');
INSERT INTO Customers VALUES(25,'Frankenversand','Peter Franken','Berliner Platz 43','München','80805','Germany');
INSERT INTO Customers VALUES(25,'Frankenversand','Peter Franken','Berliner Platz 43','München','80805','Germany');
INSERT INTO Customers VALUES(26,'France restauration','Carine Schmitt','54, rue Royale','Nantes','44000','France');
INSERT INTO Customers VALUES(27,'Franchi S.p.A.','Paolo Accorti','Via Monte Bianco 34','Torino','10100','Italy');
INSERT INTO Customers VALUES(28,'Furia Bacalhau e Frutos do Mar','Lino Rodriguez','Jardim das rosas n. 32','Lisboa','1675','Portugal');
INSERT INTO Customers VALUES(29,'Galería del gastrónomo','Eduardo Saavedra','Rambla de Cataluña, 23','Barcelona','8022','Spain');
INSERT INTO Customers VALUES(30,'Godos Cocina Típica','José Pedro Freyre','C/ Romero, 33','Sevilla','41101','Spain');
INSERT INTO Customers VALUES(31,'Gourmet Lanchonetes','André Fonseca','Av. Brasil, 442','Campinas','04876-786','Brazil');
INSERT INTO Customers VALUES(32,'Great Lakes Food Market','Howard Snyder','2732 Baker Blvd.','Eugene','97403','USA');
INSERT INTO Customers VALUES(33,'GROSELLA-Restaurante','Manuel Pereira','5ª Ave. Los Palos Grandes','Caracas','1081','Venezuela');
INSERT INTO Customers VALUES(34,'Hanari Carnes','Mario Pontes','Rua do Paço, 67','Rio de Janeiro','05454-876','Brazil');
INSERT INTO Customers VALUES(35,'HILARIÓN-Abastos','Carlos Hernández','Carrera 22 con Ave. Carlos Soublette #8-35','San Cristóbal','5022','Venezuela');
INSERT INTO Customers VALUES(36,'Hungry Coyote Import Store','Yoshi Latimer','City Center Plaza 516 Main St.','Elgin','97827','USA');
```

Sample Output 2

```
CustomerName,City,Country
Hungry Coyote Import Store,Elgin,USA
Great Lakes Food Market,Eugene,USA
```

Sample Input 3

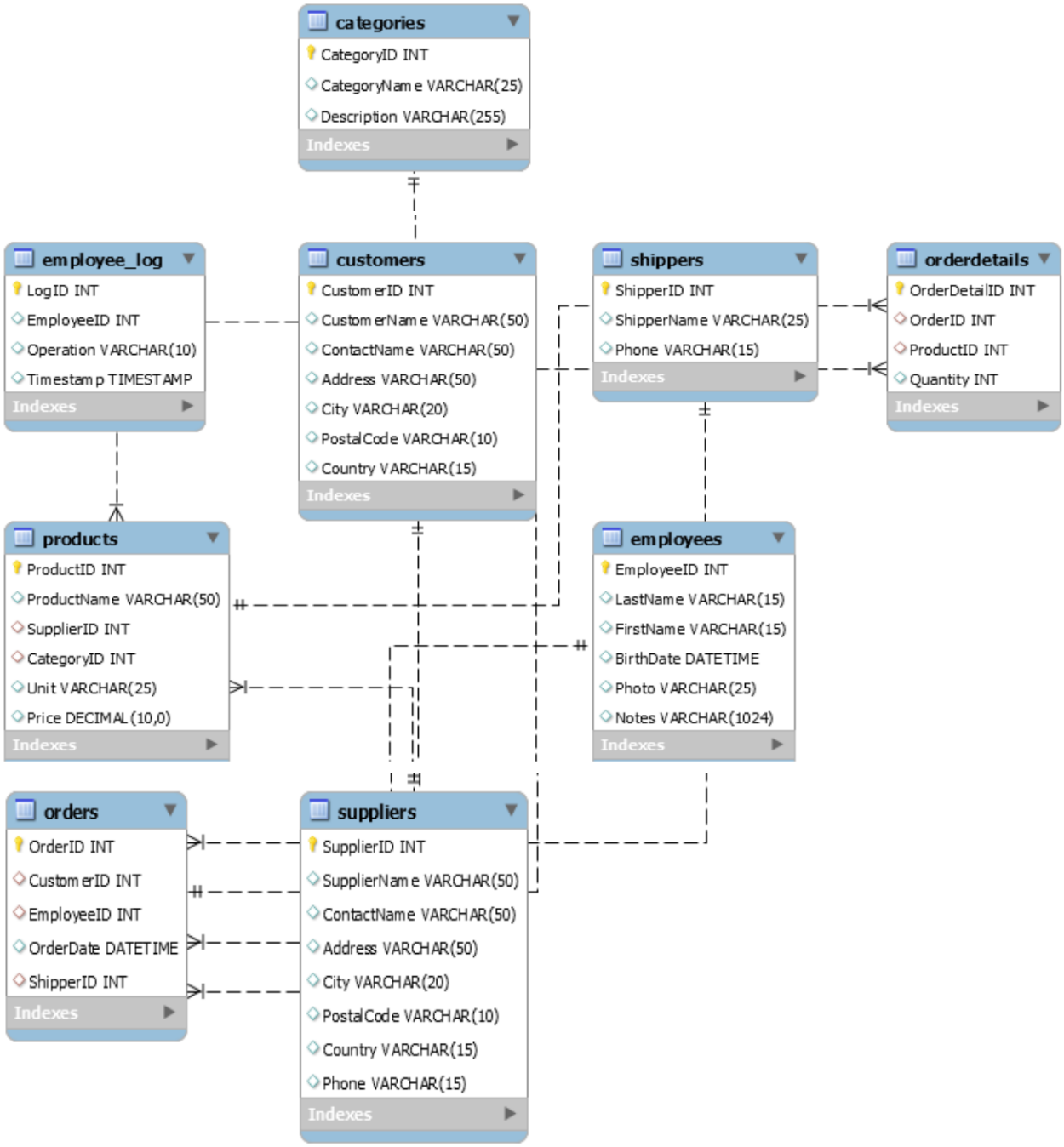
```
CREATE TEMPORARY TABLE Customers
(
    CustomerID INTEGER PRIMARY KEY AUTO_INCREMENT,
    CustomerName VARCHAR(50),
    ContactName VARCHAR(50),
    Address VARCHAR(50),
    City VARCHAR(20),
    PostalCode VARCHAR(10),
    Country VARCHAR(15)
);
INSERT INTO Customers VALUES(89,'White Clover Markets','Karl Jablonski','305 - 14th Ave. S. Suite 3B','Seattle','98128','USA');
INSERT INTO Customers VALUES(32,'Great Lakes Food Market','Howard Snyder','2732 Baker Blvd.','Eugene','97403','USA');
INSERT INTO Customers VALUES(36,'Hungry Coyote Import Store','Yoshi Latimer','City Center Plaza 516 Main St.','Elgin','97827','USA');
```

```
INSERT INTO Customers VALUES(36,'Hungry Coyote Import Store','Yoshi Latimer','City Center Plaza 516 Main St.','Elgin','97827','USA');
INSERT INTO Customers VALUES(90,'Wilman Kala','Matti Karttunen','Keskuskatu 45','Helsinki','21240','Finland');
INSERT INTO Customers VALUES(91,'Wolski','Zbyszek','ul. Filtrowa 68','Walla','01-012','Poland');
INSERT INTO Customers VALUES(92, 'Indian Spices Emporium', 'Rajesh Patel', '123 Masala Lane', 'Mumbai', '400001', 'India');
INSERT INTO Customers VALUES(93, 'Bollywood Trends', 'Priya Shah', '456 Star Street', 'New Delhi', '110001', 'India');
INSERT INTO Customers VALUES(94, 'Taj Mahal Textiles', 'Amit Kumar', '789 Fabric Road', 'Bangalore', '560001', 'India');
INSERT INTO Customers VALUES(95, 'Mumbai Electronics', 'Neha Verma', '101 Circuit Avenue', 'Mumbai', '400002', 'India');
INSERT INTO Customers VALUES(96, 'Chennai Furniture Mart', 'Arvind Singh', '234 Sofa Street', 'Chennai', '600001', 'India');
```

Sample Output 3

```
CustomerName,City,Country
White Clover Markets,Seattle,USA
Hungry Coyote Import Store,Elgin,USA
Great Lakes Food Market,Eugene,USA
```

Schema



PROBLEM 2

Develop a Java application to perform "insert" and "select" operations on movies table in a relational database using JDBC.

Below tables are already created:

a. CREATE TABLE Directors (directorId INT PRIMARY KEY AUTO_INCREMENT, directorName VARCHAR(255))
b. CREATE TABLE Movies (movieId INT PRIMARY KEY AUTO_INCREMENT, movieName VARCHAR(255), movieReleaseDate DATE, directorId INT, FOREIGN KEY (directorId) REFERENCES Directors(directorId))

Note: Insert Director table rows manually before running the code

Based on the above schema, perform the following tasks:
1. Create entity classes for Director and Movie.
Attributes for Director : directorId (type int), directorName (type String)
Attributes for Movie : movieId (type int), movieName (type String), movieReleaseDate (type Date), director (type Director). (for date import java.sql.Date;)
Create a constructor, getter and setter methods for each entity class as per the attributes mentioned above.

2.DatabaseConnection.java:(Class)

Method Signature:

public static Connection createConnection(String dbUrl, String user, String password) throws SQLException{}

1. Accepts three parameters:

- dbUrl (String): The URL of the database to which the connection should be established.
- user (String): The username required to authenticate the connection.
- password (String): The password required for authentication.

2. Uses the DriverManager.getConnection() method to create a connection to the database with the specified credentials and returns a Connection object that represents the established connection.

3. MovieDAO.java:(Interface)

Method Signature:
void insertMovie(Movie movie) throws SQLException;
List<Movie> getMoviesByName(String movieName) throws SQLException;

4.MovieDAOImpl.java this class implement MovieDAO interface:

Variables:

private Connection connection;

Method Signature:

1. public MovieDAOImpl(Connection connection) {}
Inside this constructor initialize connection

@Override
2. public void insertMovie(Movie movie) throws SQLException{}

- The insertMovie method inserts a new movie into the Movies table in the database.
- It accepts a Movie object as a parameter and uses the information (movie name, release date, and director ID) from this object to perform the insertion.
- The method prepares a SQL INSERT statement, sets the values from the Movie object using PreparedStatement, and then executes the update on the database.

@Override
3. public List<Movie> getMoviesByName(String movieName) throws SQLException {}

- The getMoviesByName method retrieves a list of movies from the database based on a partial match with the movie name.
- It uses a SELECT query with a JOIN operation between the Movies table and the Directors table. The query looks for movie names that match the provided string (with wildcard % for partial matching).
- The method constructs a list of Movie objects, each associated with a Director object, and returns the list.
- Each Movie object is populated with the details from the result set, including the movie's ID, name, release date, and director information.

---

5 .Main.java
public static void main(String[] args){}

Database requirements:
String dbUrl = "jdbc:mysql://localhost:3306/mydb";
String user = "root";
String password = "root";

- Create connection object and MovieDao Object:
  - Connection connection = DatabaseConnection.createConnection(dbUrl, user, password);
  - MovieDAO movieDAO = new MovieDAOImpl(connection);
- Read the movie name and set other values of the movie.
- create movie object and add the movie
  - Movie movie = new Movie();
  - movie.setMovieName(movieName);
  - movie.setMovieReleaseDate(Date.valueOf("2023-09-19"));
  - movie.setDirector(director);
  - movieDAO.insertMovie(movie);
- Add atleast 2 movies by calling insertMovie method and set Director id value as 1.
- Fetch all the movies by calling getMoviesByName method.
- Refer output image for clear understanding.

Notes:

- Database and Tables are already created and available for you. You don't need to create them.
- You can add data to the tables manually if you want to test the application.
- You need to read the requirment carefully and implement the same.
- You need to define the same attribute names as mentioned in the requirement.
- You need to define the same method names as mentioned in the requirement.
- You need to define the same constructors, getter and setter methods as mentioned in the requirement.

Note: First enter director id and director name in db manually.

Sample Input/Output:

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/s
Downloaded from central: https://repo.maven.apache.org/maven2/commo
Enter movie name:
Guru
Enter movie name:
Freedom India
Enter a movie name to search:
Guru
Movie ID: 1, Movie Name: Guru, Release Date: 2023-09-19, Director:
[WARNING] thread Thread[#27,mysql-cj-abandoned-connection-cleanup,5
[WARNING] thread Thread[#27,mysql-cj-abandoned-connection-cleanup,5
[WARNING] NOTE: 1 thread(s) did not finish despite being asked to
code. Although not serious, it should be remedied.
[INFO] ------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
```

PROBLEM 3

Develop a Java application to perform "insert" and "select" operations on Books table in a relational database using JDBC.
Below tables are already created

```
CREATE TABLE Authors (
authorId INT PRIMARY KEY AUTO_INCREMENT,
authorName VARCHAR(255) NOT NULL
);
CREATE TABLE Books (
bookId INT PRIMARY KEY AUTO_INCREMENT,
bookName VARCHAR(255) NOT NULL,
bookReleaseDate DATE NOT NULL,
authorId INT,
FOREIGN KEY (authorId) REFERENCES Authors(authorId)
);
```

Note:

- Insert the data into "authors" Table before running the program. Use the author name given in this table while creating books

Based on the above schema, perform the following tasks:

1. **Create entity classes for Author and Book.**
   - Attributes for Author : authorId (type int), authorName (type String)
   - Attributes for Book : bookId (type int), bookName (type String), bookReleaseDate (type Date), author (type Author).
   - Create a constructor, getter and setter methods for each entity class as per the attributes mentioned above.
   - Note:
   - Create a parameterized constructor for both the classes which accepts all the attributes as arguments.
   - You need to generate the getter and setter methods for all the attributes of both the classes as per standard java naming conventions.

2. **Create DAO interface for Book Named as BookDAO.**

Methods to be implemented in BookDAO interface:
a. **void insertBook(Book book) throws SQLException:** Method to insert a new book record in the Books table.
b. **List<Book> getBooksByName(String bookName) throws SQLException** : Method to fetch all the book records from the Books table matching the given book name.

3. **Implement the BookDAO interface in a class named as BookDAOImpl.**

a. create a private field of type Connection named as connection.
b. Create a parameterized constructor which accepts a Connection object as an argument and assigns it to the connection field. "public BookDAOImpl(Connection connection)"
c. Implement the insertBook and getBooksByName methods (override) as per the below mentioned specifications.
   - public void insertBook(Book book) throws SQLException
   - public List<Book> getBooksByName(String bookName) throws SQLException
   - For getBooksByName method, write code to select all records (along with Author name from Author table) from book table matching (like or contains) the book name then returns the result as a list of Book objects.

Note that matching can be exact or partial. For example, if the book name is "Java", then the search should return all the books with the name "Java" or "Java Programming" etc.

4. **Create Connection with DB in class named as DatabaseConnection.java. Method to be implemented in DatabaseConnection.java:**

public static Connection createConnection(String dbUrl, String user, String password) throws SQLException

5. **In Main class, Add atleast 5 books by calling insertBook method and fetch all the books by calling getBooksByName method.**

Credentials for the database connection are as follows:
1. url = "jdbc:mysql://localhost:3306/mydb"
2. user = "root"
3. password = "root"

Create the connection object and bookDAO object as follows:
   - Connection connection = DatabaseConnection.createConnection(dbUrl, user, password);
   - BookDAO bookDAO = new BookDAOImpl(connection);

Read the book details and add the books (Refer screenshot)
Search the book name and print the book details.

Notes:

- Database and Tables are already created and available for you. You don't need to create them.
- You can add data to the tables manually if you want to test the application.
- You need to read the requirement carefully and implement the same.
- Matching can be exact or partial. For example, if the search book name is "Java", then the search should return all the books with the name "Java" or "Java Programming" etc.
- First enter author details in author db manually.

Sample Input/Output:

```
Enter book name:
My Success life History
Enter book release date (YYYY-MM-DD):
2024-01-12
Enter author name:
vinoth
Enter book name:
Goal
Enter book release date (YYYY-MM-DD):
2024-10-21
Enter author name:
vinoth
Enter a book name to search:
Goal
Book ID: 4, Book Name: Goal, Release Date: 2024-10-21, Author: vinoth
```

PROBLEM 4

Imagine you are tasked with developing a simple user management system for a small company. The system should allow the company to perform basic CRUD (Create, Read, Update, Delete) operations on user data stored in a MySQL database. The company wants to interact with this system through a console application. Below are the specific requirements and scenarios you need to address:

Scenario 1: Adding a New User
- **Details**: The system should prompt the user to enter the new user's ID, name, and email. It should then check if a user with the given ID already exists. If not, it should add the new user to the database and confirm the addition.

Scenario 2: Retrieving a User by ID
- **Details**: The system should prompt the user to enter the ID of the user they want to retrieve. It should then fetch and display the user's details from the database. If the user does not exist, it should inform the user.

Scenario 3: Retrieving All Users
- **Details**: The system should provide an option to retrieve and display all users stored in the database. This list should include each user's ID, name, and email.

Scenario 4: Updating a User's Details
- **Details**: The system should prompt the user to enter the ID of the user they want to update, along with the new name and email. It should then update the user's details in the database and confirm the update. If the user does not exist, it should inform the user.

Scenario 5: Deleting a User
- **Details**: The system should prompt the user to enter the ID of the user they want to delete. It should then remove the user from the database and confirm the deletion. If the user does not exist, it should inform the user.

Implementation Details

Entity: User.java:
- Attributes: int id, String name, and String email.
- Include one empty constructor
- Declare a parameterized constructor to initialize the user details
- Includes getter and setter methods for each field.
- Overrides the toString() method to provide a formatted output

UserDaoImpl Class:
create connection object.

Define the following methods:(handle the SQLException)
- **addUser(User user)**: Adds a new user to the database after checking for duplicates
  - public boolean addUser(User user)
- **getUser(int id)**: Retrieves a user by their ID.
  - public User getUser(int id)
- **getAllUsers()**: Retrieves all users from the database.
  - public List<User> getAllUsers()
- **updateUser(User user)**: Updates an existing user's details.
  - public boolean updateUser(User user)
- **deleteUser(int id)**: Deletes a user by their ID.
  - public boolean deleteUser(int id)

Main Class:
- Provides an interactive console application to perform the above operations.
- Prompts the user for input and calls the appropriate methods in UserDaoImpl(refer Screenshot)

DatabaseConnection Class:
- Data base connectivity requirement:
  - private static final String URL = "jdbc:mysql://localhost:3306/mydb";
  - private static final String USER = "root";
  - private static final String PASSWORD = "root";
- **getConnection()** throws SQLException: Establishes a connection to the MySQL database using JDBC.

Screenshot:-

```
-----CREATE OPERATION-----
Enter the User Id
108
```

```
108
Enter the User Name
anil
Enter the user email
anil@gmail.com
User added successfully.
-----GetById OPERATION-----
Enter the Id
109
User with ID 109 not found.
----All User-----
User{id=100, name='Prasad', email='prasad@gmail.com'}
User{id=101, name='Sushil', email='sushil@gmail.com'}
User{id=102, name='Aayansh', email='aayansh@gmail.com'}
User{id=103, name='Anil', email='anil@gmail.com'}
User{id=105, name='Abhishek', email='abhishek@gmail.com'}
User{id=106, name='Ajay', email='ajay@gmail.com'}
User{id=107, name='Vijay', email='vijay@gmail.com'}
User{id=108, name='anil', email='anil@gmail.com'}

-----UPDATE OPERATION-----
Enter the Id
107
Enter the name
Amit
Enter the email
amit@gmail.com
User updated successfully.
-----DELETE OPERATION-----
Enter the Id
```